

GIT INIT AND REMOTE REPOSITORY SETUP

Git Init - Used to create and initialize a new Git repository in the local system. It creates a hidden .git folder that helps Git track files, commits, and version history in the project directory.

HANDS-ON GIT INIT AND REMOTE REPOSITORY SETUP

Screenshot: Create New Repository

- Open GitHub
- Click New repository
- Enter repository name
- Click Create repository

The screenshot shows the 'Create a new repository' page on GitHub. It is divided into two sections: 'General' and 'Configuration'.

General Section:

- Owner:** A dropdown menu showing 'jaisurya003'.
- Repository name:** A text input field containing 'jai-repo'. Below the field, a green checkmark indicates 'jai-repo is available'.
- Description:** A text input field with a character count '0 / 350 characters'.

Configuration Section:

- Choose visibility:** A dropdown menu set to 'Public'.
- Add README:** A toggle switch set to 'Off'.
- Add .gitignore:** A dropdown menu set to 'No .gitignore'.
- Add license:** A dropdown menu set to 'No license'.

At the bottom right of the form is a green button labeled 'Create repository'.

Screenshot: Initialize Git Repository

- Open Git Bash
- Execute `cd Downloads` (to move into Downloads folder)
- Execute `mkdir repository-name` (directory name same as created GitHub repository)
- Execute `cd repository-name` (to move into project folder)
- Execute `git init` (to initialize new Git repository)
- Execute `ls -la` (to view all files and folders)
- .git folder displayed after repository initialization

```
MINGW64:/c:/Users/JAISURYA/downloads/jai-repo
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~ (master)
$ cd downloads

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads (master)
$ mkdir jai-repo

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads (master)
$ cd jai-repo

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git init
Initialized empty Git repository in C:/Users/JAISURYA/Downloads/jai-repo/.git/

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ ls -la
total 164
drwxr-xr-x 1 JAISURYA 197121 0 May 17 18:35 ./
drwxr-xr-x 1 JAISURYA 197121 0 May 17 18:33 ../
drwxr-xr-x 1 JAISURYA 197121 0 May 17 18:35 .git/
```

Screenshot: Add and Commit Files to Repository

- Execute touch app.py (to create a new Python file)
- Execute git add . (to add files to staging area)
- Execute git status (to check repository status)
- Execute git commit -m "first commit" (to save changes in local repository)
- Execute ls -la (to view all files and folders)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ touch app.py

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git status -s
?? app.py

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git add .

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git status -s
A app.py

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git commit -m "first"
[master (root-commit) 9059d38] first
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 app.py

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ ls -la
total 164
drwxr-xr-x 1 JAISURYA 197121 0 May 17 18:36 ./
drwxr-xr-x 1 JAISURYA 197121 0 May 17 18:33 ../
drwxr-xr-x 1 JAISURYA 197121 0 May 17 18:36 .git/
-rw-r--r-- 1 JAISURYA 197121 0 May 17 18:36 app.py
```

Screenshot: Configure Main Branch

- Execute git remote -v (to check configured remote repositories)
- No current remote repository configured
- Execute git branch -M main (to rename master branch to main branch)
- Execute git branch (to view available branches)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git remote -v

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (master)
$ git branch -M main

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git branch
* main
```

Screenshot: Connect Local Repository to GitHub

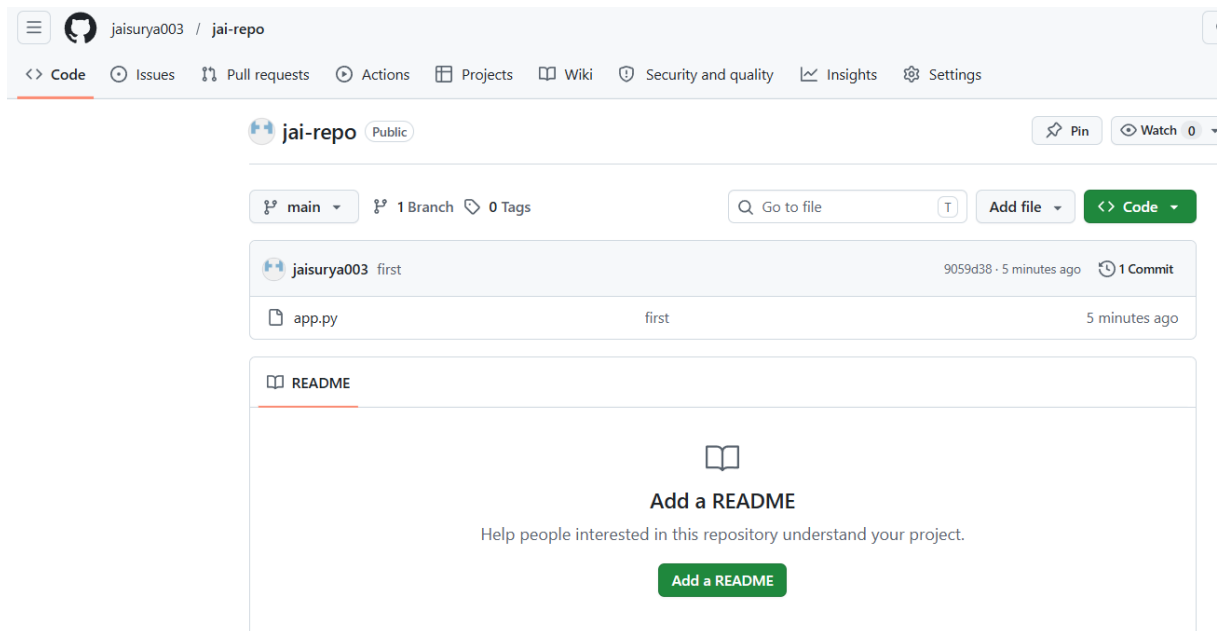
- Execute `git remote add origin https://github.com/username/repository.git` (to connect local repository with GitHub repository)
- Execute `git push -u origin main` (to push local repository to GitHub main branch)
- Main branch successfully pushed to remote repository
- Local branch connected with remote tracking branch `origin/main`

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git remote add origin https://github.com/jaisurya003/jai-repo.git

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 206 bytes | 51.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jaisurya003/jai-repo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Screenshot: Verify Files in GitHub Repository

- Refresh the GitHub repository page
- `app.py` file displayed successfully in repository



The screenshot displays the GitHub interface for a repository named 'jai-repo' owned by 'jaisurya003'. The repository is marked as 'Public'. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security and quality, Insights, and Settings. Below the repository name, it shows 'main' as the selected branch, with '1 Branch' and '0 Tags' indicated. A search bar and 'Add file' button are present. The commit history shows a single commit by 'jaisurya003' with the message 'first', committed 5 minutes ago. The file 'app.py' is listed as part of this commit. Below the commit list, there is a section for 'README' with a prompt to 'Add a README' to help people understand the project.

GIT RESET

- Git Reset - Used to undo commits or changes by moving the HEAD pointer to a previous commit in the repository.
- It is mainly used before pushing changes to the remote repository.

Types of Git Reset

1. **git reset --soft <HEAD ID>**: Moves the HEAD to the specified commit, but keeps both the staging area and working directory unchanged.
2. **git reset --mixed <HEAD ID>**: Moves the HEAD to the specified commit and resets the staging area, but keeps the working directory unchanged.
3. **git reset --hard <HEAD ID>**: Moves the HEAD to the specified commit, resets the staging area, and permanently removes changes from the working directory.

HANDS-ON FOR GIT RESET

Screenshot: Create Multiple Commits

- Execute vi app.py (to edit the existing file)
- Execute git status -s (to view short status output)(Red color indicates modified file status)
- Execute git add . (to add modified file to staging area)
- Execute git commit -m "second commit" (to save second change in repository)
- Execute vi app.py (to modify the file again)
- Execute git add . (to add modified changes to staging area)
- Execute git commit -m "third commit" (to save third change in repository)

```
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
9059d38 (HEAD -> main, origin/main) first
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ vi app.py
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git status -s
M app.py
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git commit -am "second"
git: 'commit' is not a git command. See 'git --help'.
The most similar command is
commit
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git commit -am "second"
warning: in the working copy of 'app.py', LF will be replaced by CRLF the next time Git touches it
[main 084448b] second
1 file changed, 1 insertion(+)
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ vi app.py
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git commit -am "third"
warning: in the working copy of 'app.py', LF will be replaced by CRLF the next time Git touches it
[main 28259de] third
1 file changed, 1 insertion(+)
JAI SURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
28259de (HEAD -> main) third
084448b second
9059d38 (origin/main) first
```

Screenshot: Create Multiple Commits

Repeat the same process to create 6 commits in the repository

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ vi 2.py

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git commit -am "six"
warning: in the working copy of '2.py', LF will be replaced by CRLF the next time Git touches it
[main 2187489] six
1 file changed, 1 insertion(+)

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
2187489 (HEAD -> main) six
2ddc662 five
126a5e3 four
28259de third
084448b second
9059d38 (origin/main) first
```

Screenshot: Perform Soft Reset

- Execute git log --oneline (to view commit history in one line format)
- Execute git reset --soft <fifth-commit-id> (to move HEAD to fifth commit and keep changes in staging area)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
2187489 (HEAD -> main) six
2ddc662 five
126a5e3 four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --soft AC

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --soft 2ddc662
```

Screenshot: Verify Soft Reset Changes

- HEAD moved from sixth commit to fifth commit
- Execute git status (to check repository status)
- Changes remain unchanged in staging area after soft reset

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --soft 2ddc662

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
2ddc662 (HEAD -> main) five
126a5e3 four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git status -s
M 2.py
```

Screenshot: Perform Mixed Reset

- Execute git log --oneline (to view commit history in one line format)
- Execute git reset --mixed <fourth-commit-id> (to move HEAD to fourth commit and remove changes from staging area)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
2187489 (HEAD -> main) six
2ddc662 five
126a5e3 four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --mixed 126a5e3
```

Screenshot: Verify Mixed Reset Changes

- HEAD moved from sixth commit to fourth commit
- Execute git status -s (to view short status output)
- Red color indicates modified unstaged changes after mixed reset

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
126a5e3 (HEAD -> main) four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git status -s
?? 2.py
```

Screenshot: Perform Hard Reset

- Execute git log --oneline (to view commit history in one line format)
- Execute git reset --hard <third-commit-id> (to move HEAD to third commit and remove all changes permanently)

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
126a5e3 (HEAD -> main) four
28259de third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --hard 28259de
```

Screenshot: Verify Hard Reset Changes

- HEAD moved to third commit
- Execute git status (to check repository status)
- No red color displayed after hard reset because changes were permanently removed

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git reset --hard 28259de
HEAD is now at 28259de third

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git log --oneline
28259de (HEAD -> main) third
084448b second
9059d38 (origin/main) first

JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/downloads/jai-repo (main)
$ git status -s
?? 2.py
```

GIT REFLOG

Git Reflog - Used to view the history of HEAD movements and recent Git actions such as commits, resets, checkouts, and merges in the repository.

```
JAISUR/ASLAPTOP-69609304 MINGW64 ~/Downloads/jai-pepo (main)
$ git reflog --oneline
084448b (HEAD -> main) HEAD@{0}: reset: moving to 084448b
28259de HEAD@{1}: reset: moving to 28259de
126a5e3 HEAD@{2}: reset: moving to 126a5e3
2187489 HEAD@{3}: reset: moving to 2187489
2ddc662 HEAD@{4}: reset: moving to 2ddc662
2187489 HEAD@{5}: commit: six
2ddc662 HEAD@{6}: commit: five
126a5e3 HEAD@{7}: commit: four
28259de HEAD@{8}: commit: third
084448b (HEAD -> main) HEAD@{9}: commit: second
9059d38 (origin/main) HEAD@{10}: Branch: renamed refs/heads/master to refs/heads/main
9059d38 (origin/main) HEAD@{12}: commit (initial): first
```

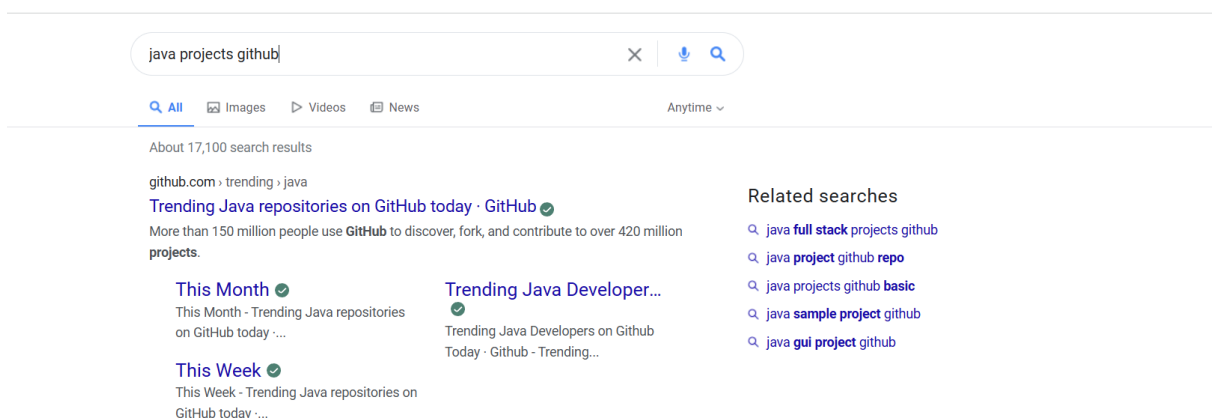
GIT FORK

- Git Fork - Used to create a personal copy of another user's GitHub repository into your own GitHub account for independent development and collaboration.
- It allows users to modify, experiment, and contribute to projects independently without affecting the original repository.
- Forking is commonly used in open-source collaboration and pull request workflows.

HANDS-ON FOT FORK

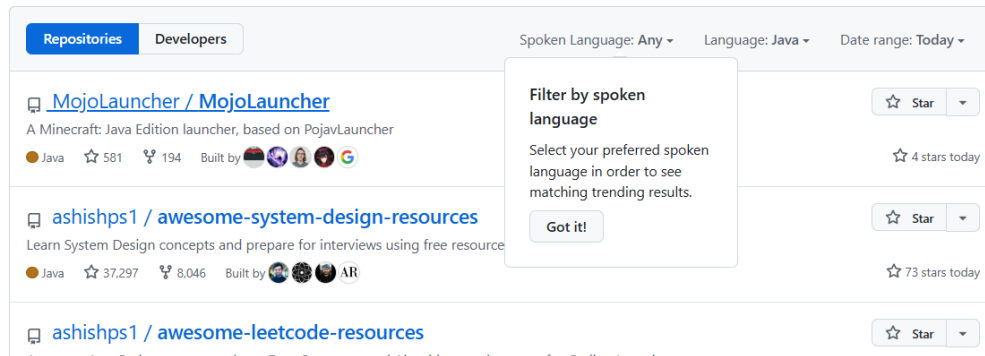
Screenshot: Search GitHub Java Project

- Open Google
- Search for Java project in GitHub
- Open a public GitHub Java project repository



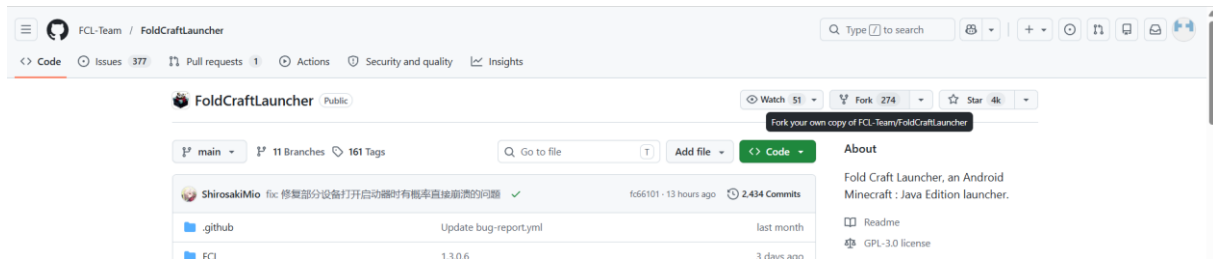
Screenshot: Open Public GitHub Repository

- Open any public GitHub account
- View available repositories in the account



Screenshot: Fork Repository

- Click Fork
- Start creating a copy of the repository into personal GitHub account



Screenshot: Create Forked Repository

- Enter repository name
- Click Fork

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Required fields are marked with an asterisk (*).

Owner * / Repository name *

fork is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

66 / 350 characters

- ☒ Copy the `main` branch only
- Contribute back to FCL-Team/FoldCraftLauncher by adding your own branch. [Learn more.](#)

You are creating a fork in your personal account.

Create fork

Screenshot: Forked Repository Created Successfully

- Repository copied successfully into personal GitHub account
- Forked repository available for independent development and changes

